



Lecture 4 - overview

epcc

- Outline
 - This lecture we'll look at some advanced features of Fortran 2008
 - Derived data types
 - Pointers
 - File I/O
 - Formatting output

Derived data types

- Fortran 2008* allows the use of derived data types
- In many algorithms data types can be grouped together to form an aggregate structure
- Often useful to be able to manipulate objects that are more sophisticated than the intrinsic types
- Allows linked data structures, lists, trees etc
- Derived data types are often needed in Coarray Fortran
- Imagine we wish to specify objects representing persons
 - Each person is uniquely distinguished by a name, age and ID number
- We can define a corresponding “person” data type as follows:

* allowed since Fortran 90

Derived data types

```
type person
  character (len=10) :: name
  real              :: age
  integer           :: id
end type person
```

- To declare a variable of type person use the syntax
`type(person) :: you, me`
- Can assign values to the variable `you` using
`you = person("Joe Bloggs", 21, 1234)`
- `you` is a variable containing 3 elements: `name`, `age` & `id`

Derived data types

- The elements of derived type **person** may be accessed by using the variable name and the element name separated by the % character, e.g.

```
you%name ! contains the name of you  
you%age  ! contains the age of you  
you%id   ! contains the id of you
```

- % is known as the *component selector*
- Can perform computations using derived type variables as follows:
 - Difference in age between variables **you** and **me**

```
real :: age_diff  
age_diff = you%age - me%age
```

Pointers

- Pointers were included in the Fortran 90 standard
 - Included in Fortran 2008 and useful for Coarray Fortran
- Similar but not identical to pointers used in C/C++/Java
- In other languages a pointer stores an *address* rather than the actual value
- In Fortran, a pointer is an *alias* for the variable it points to
 - Pointer variables do not store addresses, instead they evaluate to the variable that they point to
 - No need to dereference pointer variables in Fortran
- Fortran pointers can also point to a section of data structure, e.g. to a row or column of an array or part of a derived data type

Pointers

- To declare a pointer variable, add the keyword, **pointer**, to the variable declaration, e.g.
`real, pointer :: f`
- We also need to identify the variables to which **f** can point. Use the keyword, **target**, to do this, e.g.
`real, target :: a`
- Assignment of pointer variables is achieved using the pointer assignment operator, **=>**, e.g.
`f => a`
- The memory location associated with **a** is now also the memory location associated with **f**
 - If the value of **a** changes then the value of **f** also changes and vice versa as they now share the same memory location
- Can use **associated** to test whether a pointer points to something

Pointers – simple example

```
program pointer_example
  implicit none
  real, target :: a=3.141, b=1.414, c=0.866    ! initial values
  real, pointer :: r, s                        ! pointers to real values

  ! Pointer associations result in a = r = 3.141 and b = s = 1.414
  r => a
  s => b

  ! Conventional assignments, variables a, b, r, and s will be 3.141
  s = r

  ! Can use pointer association to set r = c = 0.866 without changing
  ! a, b, or s.
  r => c

end program pointer_example
```

Pointers and arrays

- Can also have pointers to arrays

```
real, dimension(8), target :: mydata
real, dimension(:), pointer :: index
```

- Pointer arrays just have a rank
 - They are declared without any bounds
 - Bounds are picked up from the target array
 - Target and pointer ranks must match

```
mydata = (/ (i, i=1,8) /)
index => mydata(1:8:2)
write(*,*) "Size of target array = ", size(mydata) ! 8
write(*,*) "Size of pointer array = ", size(index) ! 4
```

Simple I/O – write statement

- Can use write and read statements to access standard input (i.e. screen and keyboard)

```
write(*,*) "This text will appear on the screen"
write(*,*) "Input temperature (C)"
read(*,*) temperature ! Reads value input via
                      ! the keyboard and assigns
                      ! to variable temperature
```

- Multiple values can be read in from a single line

```
write(*,*) "Input 3 results"
read(*,*) result1, result2, result3
```

Simple I/O – write statement

- Use `write()` statement

```
write ([unit =] unit, &  
      [fmt =] format_string ...) [list]  
– can take default write (*,*)  
– i.e., standard output and free format
```

- To write to an external file

```
open (unit = 20, file = "file.dat", &  
      form = "formatted", action = "write")  
write (unit = 20, fmt = *) [list]  
close (unit = 20, status = "keep")
```

- Input is via `read()`

– e.g. `read(*,*) temperature` to read the value of `temperature` from the keyboard

Simple I/O – unknown file length

- To read in values from file until the end of the file is reached:

```
integer :: i, icount = 0  
integer, parameter :: maxlen=500  
real, dimension(maxln) :: a  
open(unit=10, file="temps.dat", status="old", action="read")  
do i = 1, maxlen  
    read(10,*,end=100)a(i)  
    icount = icount + 1  
end do  
100 continue          ! 100 is a label  
close(10)  
write(*,*)"No. of lines read in from file =",icount
```


Formatted I/O

- Often we want to control the format of the output (or input) data
 - A length calculated by a program to be 5.234523 m is nonsensical if the original measurement was made using a ruler accurate to 1 mm (5.235 m would be far more suitable)
- This can be achieved using the **fmt** specifier or the **format** statement, e.g.
- The syntax of the **fmt** specifier is as follows:

```
write(*, fmt="(i2,2x,f5.1,1x,a4)") i,temp,name
```

- Writes to the screen 3 variables according to the specified format “an integer of length 2 digits (i2), 2 spaces (2x), a real number of total length 5 digits with 1 digit after the decimal point (f5.1), 1 space (1x), a character of length 4 digits (a4)”

Formatted I/O

- The syntax of the **format** statement is as follows:

```
write(*,10) i, temp, name
10 format (i2,2x,f5.1,1x,a4)
```

 - Where 10 is a statement label
- The first form is usually used when there is only a single reference in the format specification
 - Can improve code readability
- The second form is usually used when there are several references or the form is complex
- Formatting is sometimes known as I/O editing. The I/O is controlled via edit descriptors
 - **i2**, **2x**, **f5.1**, **1x** and **a4** are examples of edit descriptors

Edit descriptors

- The most commonly used edit descriptors are listed

Format	Description	Example syntax	Example numeric
iw	w chars of integer data	i5	---10
fw.d	w chars of real data (d dec. pl.)	f12.4	-----12.4568
ew.d	w chars of real data (d dec. pl.)	e14.6	--0.124568E+02
lw	w chars of logical data	l4	---T
a[w]	w chars of character data	a4	Fred
nx	skip n chars (n spaces)	3x	---

- “-” indicates a blank space
- “w” determines the total number of characters the data spans
- “d” gives the no. of decimal places – number is *rounded* not truncated
- If a record does not occupy the entire length allocated in the format statement then it is padded with blanks

Edit descriptors

- If a record does not fit into the amount of space allocated via the formatting then *’s are displayed instead, e.g.

```
integer :: j = 123456
```

```
write(*,"(i2)") j
```

- Will display ** as the output
 - Need to use *at least* i6 to display j correctly
- When using format specifications it is better to make them slightly larger than required
 - Frustrating when a long program run contains ****’s in the output!
- Further details are given in Metcalf and Reid

Unformatted I/O

- So far have looked only at formatted I/O
- Formatted I/O has several drawbacks
 - An overhead is incurred for converting between the two forms (internal to external)
 - Typically the data will be rounded; overhead, loss of precision
 - Generally formatted data (ASCII) occupies more disk space than the unformatted (binary) version
- Unformatted I/O allows the internal structure of the data to be preserved

Unformatted I/O

- Gives the internal representation of a value written *exactly* as it exists on a storage medium
 - Can be read directly without round-off or conversion overheads
- Syntax

```
open(unit=10, file="unformatted.dat",& action="write",
form="unformatted" )

. . .
do i = 1, imax
  write(10, err=100) a(i)
end do
100 continue
close(10)
```
- No complex format statements required

Formatted v unformatted I/O

- When should unformatted I/O be used?
 - Almost all the time!
 - If speed / size is an issue
 - If records are generated by a program on one computer and read back on the same computer (or another computer using the same internal number representation)
- Use formatted I/O
 - When computers with different internal number representations are used to write and read the same data
 - If the data need to be visualised (tabular or graphical)

Unit numbers

- Files are accessed by unit number
- Program can connect named file to a unit number or use default value
 - E.g. CDROM drive could be associated with unit number 15
- Unit number (integer) must not be negative and is typically in the range 1-99
- Can be specified as
 - An integer constant, e.g. 10
 - An integer expression, e.g. `nunit`, `nunit + 1`
 - An asterisk `*` for the default value (usually keyboard or screen)
- Unit numbers 5 and 6 are assumed special
 - 5 usually corresponds to **stdin** - usually the keyboard
`read(5,*) niter`
 - 6 usually corresponds **stdout** – usually the screen
`write(6,*) "Hello World!"`

Summary

- This session we have looked at some advanced features of Fortran 2008 including:
 - Derived data types
 - Pointers
 - File I/O
 - Differences between formatted/unformatted I/O
 - Edit descriptors and format statements/ specifiers can be used to control the appearance of output
 - For performance or portability unformatted I/O is usually best